

Elyos Interview 2.0

 [interview.md](#)

Elyos AI Technical Interview

Overview

This is a hybrid technical assessment. You'll complete a take-home exercise, record a video walkthrough, and (if successful) join us for an on-site pairing session.

We're looking for engineers who can independently own complex systems, make sound technical decisions, and ship high-quality code without constant guidance.

Format:

1. **Take-home** (45-90 min): Build a streaming CLI with quirky real-world APIs
2. **Loom video** (10-15 min): Demo your work and explain what you discovered
3. **On-site** (2.5 hours): Pair programming, architecture discussion, technical deep-dive

Part 1: Take-Home Implementation

The Task

Build a command-line chat application that:

1. Accepts text input from the user
2. Sends input to an LLM (OpenAI, Anthropic, or similar)
3. **Streams** the response back to the terminal in real-time
4. Supports **tool calling** with two APIs:
 - A weather API (usually fast, ~200ms)
 - A "research" API (slow, 3-8 seconds)

5. Handles **pending states** — show the user something is happening during slow tool calls
6. Supports **cancellation** — user can interrupt a long-running operation (Ctrl+C or similar)
7. **Handles the APIs gracefully** — these are real-world APIs with real-world quirks

The Catch

The APIs you'll integrate with are intentionally imperfect. Like real production APIs, they have undocumented behaviors, edge cases, and occasional failures. Part of this challenge is discovering these behaviors and handling them appropriately.

We will not tell you what the quirks are. You should:

1. Build the integration
2. Discover unexpected behaviors through testing
3. Handle them gracefully
4. Document what you found

This mirrors real-life IC work: you rarely get perfect APIs with complete documentation.

Technical Requirements

Language: Python or TypeScript preferred (we use both). Other languages acceptable if you're significantly stronger in them but we strongly recommend dynamic typing languages (e.g. Ruby).

LLM Integration: Use the provider's official SDK. You should understand:

- [Streaming responses](#)
- [Function/tool calling](#)

For the on-site (familiarize yourself, no implementation needed for take-home):

- [Deepgram Streaming SDK](#) - for speech-to-text
- [ElevenLabs Streaming SDK](#) - for text-to-speech

Note: We can provide API keys for all services, and most offer free trials.

APIs:

```
# Get weather for a location
curl -H "X-API-Key: <provided>" \
  "https://elyos-interview-907656039105.europe-west2.run.app/weather?locat
```

```
# Research a topic (slow: 3-8 seconds)
curl -H "X-API-Key: <provided>" \
  "https://elyos-interview-907656039105.europe-west2.run.app/research?topi
```

Basic API documentation:

Endpoint	Method	Parameters	Returns
/weather	GET	location (string)	Weather data for the location
/research	GET	topic (string)	Research summary on the topic

Both endpoints require the `X-API-Key` header.

The `/research` endpoint takes 3-8 seconds to respond. This simulates real-world scenarios like database queries or AI processing.

Note: This is the complete official documentation. Any other behaviors you observe are part of the challenge.

Starter Template (Python)

```
import asyncio

async def get_user_input() -> str:
    """Get input from user."""
    pass

async def call_llm(user_input: str, conversation_history: list):
    """Send input to LLM, handle tool calls, yield streaming response."""
    pass

async def get_weather(location: str) -> dict:
    """Fetch weather from API (~200ms)."""
    pass

async def research_topic(topic: str) -> dict:
    """Research a topic (3-8 seconds). Should be cancellable."""
    pass

async def main():
    conversation_history = []

    while True:
        user_input = await get_user_input()
        if user_input.lower() in ['quit', 'exit', 'q']:
            break

        # How do you handle cancellation while streaming?
        # How do you show pending state during slow tool calls?
```

```

    async for chunk in call_llm(user_input, conversation_history):
        print(chunk, end='', flush=True)
    print()

if __name__ == "__main__":
    asyncio.run(main())

```

Key challenge: While a slow tool call is running, the user should:

1. See that something is happening (e.g., "Researching solar energy...")
2. Be able to cancel and return to the prompt
3. See partial results if the LLM was mid-stream when they cancelled

Tool Definition Template

```

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather for a city. Fast response.",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "City name, e.g. London, Tokyo"
                    }
                }
            },
            "required": ["location"]
        }
    },
    {
        "type": "function",
        "function": {
            "name": "research_topic",
            "description": "Research a topic in depth. Takes 3–8 seconds.",
            "parameters": {
                "type": "object",
                "properties": {
                    "topic": {
                        "type": "string",
                        "description": "Topic to research, e.g. 'solar ene"
                    }
                }
            },
            "required": ["topic"]
        }
    }
]

```

```
}  
]
```

What We Expect

Must have:

- Working streaming (response appears incrementally, not all at once)
- Both tool calls working (weather + research)
- **Pending state indication** during slow research calls (user knows something is happening)
- **Cancellation support** — user can interrupt a long-running research call
- **Graceful API handling** — your code should handle whatever the APIs throw at it
- **Conversation history maintained across turns**
- Code a competent engineer could understand in 5 minutes

Discovery & documentation:

- Keep notes on any unexpected API behaviors you encounter
- Your handling doesn't need to be perfect, but it should be intentional
- In your Loom video, you'll walk through what you discovered

Nice to have (don't over-engineer):

- Graceful handling of partial results on cancellation
- Clean separation of concerns
- Spinner or progress indicator during pending state

We don't need:

- Tests (one or two is fine, not a full suite)
- Perfect abstractions or design patterns
- Documentation beyond brief comments
- A fancy UI

Target size: ~150-250 lines of focused code. If you're past 400 lines, you're probably over-engineering.

Example Interactions

You: What's the weather in Tokyo?

Assistant: [calls get_weather]

The weather in Tokyo is currently 22°C and sunny.

You: Research renewable energy trends
Assistant: [calls research_topic]
Researching renewable energy trends... (Ctrl+C to cancel)
[3-8 seconds pass]
Based on my research, here are the key trends in renewable energy...

You: Research quantum computing
Assistant: [calls research_topic]
Researching quantum computing... (Ctrl+C to cancel)
[user presses Ctrl+C after 2 seconds]
Research cancelled.

You:

Part 2: Loom Video

Record a 10-15 minute video covering:

Section	Time	What to Cover
Demo	3-4 min	Show your app working: (1) weather query, (2) research query with pending state, (3) cancelling a research query mid-flight
API Discovery	3-4 min	What unexpected behaviors did you find in the APIs? How did you discover them? How did you handle each one?
Code Walkthrough	3-4 min	Walk through your implementation. Focus on: How does streaming work? How did you implement cancellation?
Trade-offs	2-3 min	What's one decision you made that has trade-offs? What alternative did you consider?
Self-Critique	1-2 min	What would you change with more time? What's the weakest part?

On API Discovery:

This is the most important section. We want to understand:

- What did you observe that wasn't in the documentation?
- How did you figure out what was happening?
- What was your reasoning when deciding how to handle it?
- Did anything surprise you?

Being thorough here matters more than having "perfect" code.

Tips:

- Don't rehearse to perfection—we want to see how you think, not a polished presentation
- It's fine to say "I'm not sure about X, but here's how I'd find out"
- If you didn't find all the quirks, that's okay—explain what you did find
- Showing a bug and explaining how you'd fix it is better than hiding it

What We Evaluate

We're looking at three things:

1. Implementation Quality

- Does streaming work properly?
- Do both tool calls work?
- Is there a clear pending state during slow operations?
- Does cancellation actually cancel?
- Are errors handled gracefully?
- Is the code clear and maintainable?

2. API Discovery & Handling

- Did you find unexpected API behaviors?
- Did you handle them gracefully?
- Can you explain what you discovered and why you handled it that way?

3. Communication

- Is your Loom video clear and organized?
- Do you discuss trade-offs in your decisions?
- Are you honest about weaknesses and areas for improvement?

Note: The API discovery section is particularly important. We're looking for engineers who investigate systems thoroughly rather than assuming everything works as documented.

On-Site Session (2.5 hours)

If your take-home meets our bar, we'll invite you for an on-site session:

Part 1: Code Discussion (30-45 min)

We'll review your take-home together:

- Walk us through how you implemented cancellation
- What happens if the user cancels mid-stream while the LLM is responding?
- Tell us more about [specific API quirk]—how did you discover it? Why did you handle it that way?
- Were there any quirks you noticed but didn't have time to handle properly?
- Let's discuss how you'd add retry logic with backoff

What we're evaluating: Depth of understanding, investigative thinking, receptiveness to feedback

Part 2: Pair Programming (75 min)

We'll extend your implementation together. Possible directions:

- Add real-time audio input (we'll provide the STT integration)
- Handle concurrent tool calls
- Add conversation memory/context management

You drive, we navigate. We'll give you unfamiliar APIs and see how you work through them.

What we're evaluating:

- How you approach unfamiliar problems
- How you ask clarifying questions
- Code quality under time pressure
- Collaboration style (do you explain your thinking?)

Part 3: System Design Discussion (45 min)

We'll discuss how to scale the system:

- "How would you handle 100 concurrent voice calls?"
- "What breaks first? How would you monitor it?"
- "Walk me through a request from mic input to speaker output—where's the latency?"

This is a discussion, not a whiteboard test. We want to see how you think about systems, not perfect answers.

What we're evaluating:

- Systems thinking
- Awareness of operational concerns (monitoring, debugging, failure modes)
- Ability to reason about trade-offs at scale

What We're Looking For (Summary)

Strong engineers at Elyos:

- Own complex systems end-to-end without hand-holding
- **Investigate before assuming** — test APIs, read responses carefully, notice oddities
- Comfortable with async/concurrent code and understand the pitfalls
- Handle real-world messiness: cancellation, timeouts, partial failures, quirky APIs
- Write code that other engineers can maintain
- Make reasonable decisions quickly, perfect decisions when it matters
- Communicate technical concepts clearly
- Know what they don't know

What to avoid:

- Didn't test the APIs beyond happy path (missed obvious quirks)
- Can't explain their own code
- Cancellation doesn't actually work or leaves resources hanging
- No error handling or "happy path only" thinking
- Blames the API for issues instead of handling them
- Defensive when discussing trade-offs or weaknesses

Logistics

API Keys: We'll provide keys for the weather API. Use your own LLM API key (most providers have free tiers) or let us know if you need one.

Timeline: Please complete the take-home within 1 week of receiving it. The Loom video should be submitted with your code.

Submission: Send us:

1. Link to code (GitHub repo, zip file, or similar)
2. Link to Loom video
3. If you used an AI assistant (Claude Code, Cursor, Copilot, etc.), please include your session transcript. We want to understand how you use these tools, not whether you do. For example, in Claude Code you can export your session with the `/export` command.

Questions? Email us. Clarifying questions are encouraged—senior engineers ask good questions.